

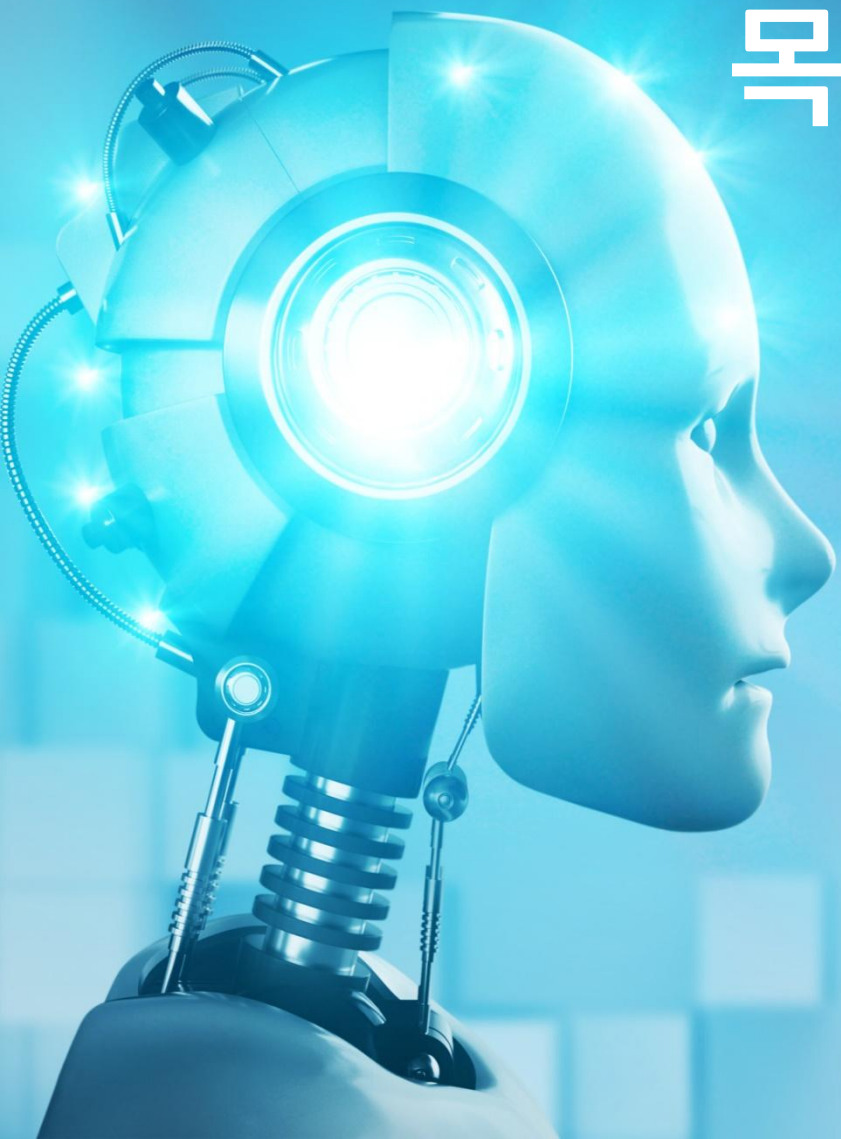
인공지능(AI) 개발자 양성 과정

5
차시

LLM 기본

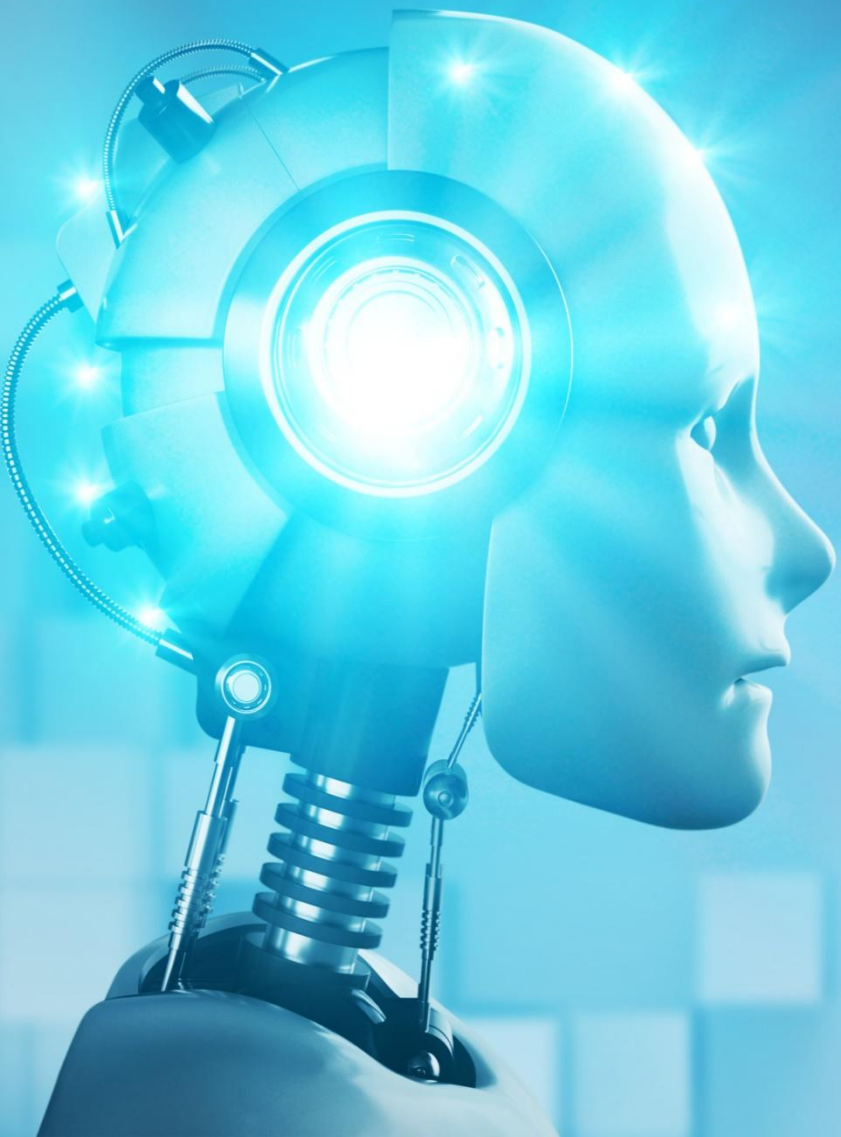
휴먼교육센터 안홍근 강사





목차 Contents

1. LLM 기초



1 LLM 기초

1. 거대언어모델(LLM) 정의
2. 트랜스포머(Transformer)
3. 국내외 거대언어모델(LLM)
4. 허깅페이스(Hugging Face)
5. 랭체인(LangChain)
6. RAG(검색증강생성)

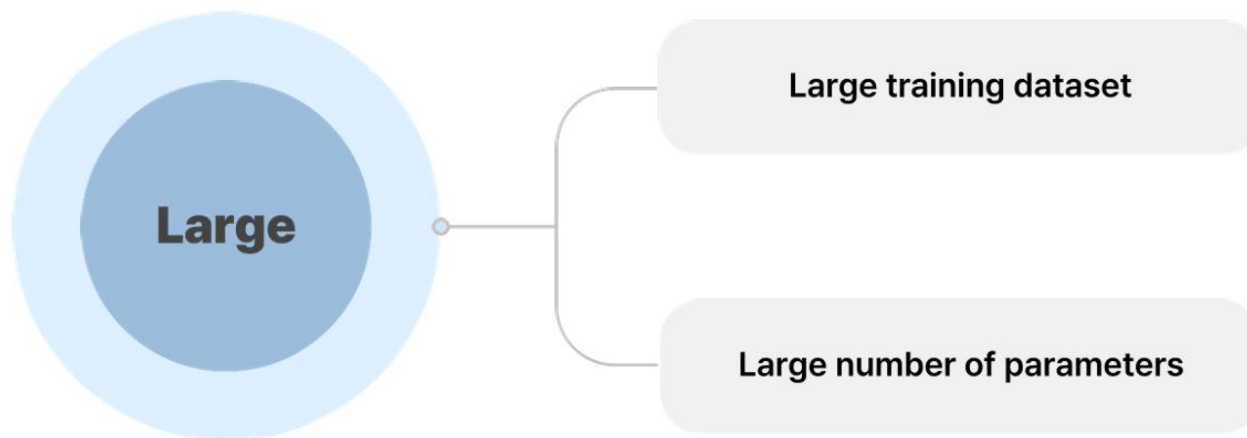
1. 거대언어모델(LLM) 정의

1.1 LLM (Large Language Model)이란?

- ☑ 많은 양의 텍스트 데이터를 활용해 일반적인 수준의 언어이해가 가능하도록 사전 학습을 시킨 언어모델
- ☑ 다량의 텍스트 데이터를 기반으로 문맥을 이해하고, 자연스러운 언어를 생성할 수 있는 딥러닝 기반 모델

특징

- 대규모 데이터 학습 : 인터넷에 공개된 웹페이지, 도서, 논문 등 다양한 자료를 바탕으로 학습
- 문맥 이해 : 단순히 단어를 나열하는 것이 아니라, 문맥과 의미를 고려한 결과를 생성
- 다양한 작업 수행 가능 : 질의응답, 글쓰기, 번역, 요약, 코딩, 추천 시스템 등 폭넓은 응용 가능
- 사전 학습(Pre-training) 및 미세 조정(Fine-tuning) : 사전학습으로 기본적인 언어 이해 능력을 익힌 후, 특정 작업에 맞게 미세 조정 가능



1. 거대언어모델(LLM) 정의

1.2 NLP(자연어 처리) ▶ 1.2.1 정의 및 특징

- ☑ NLP(Natural Language Processing)는 컴퓨터가 인간의 언어를 이해하고, 처리하며, 생성할 수 있도록 하는 기술
- ☑ 이를 통해 컴퓨터는 인간의 자연어(한국어, 영어 등)를 분석하거나, 이를 바탕으로 유용한 결과를 생성할 수 있음

특징

- 인간 언어의 복잡성 : 다양한 표현 방식과 문법 구조, 모호성 문제를 처리해야 함
- 비구조적 데이터 처리 : 텍스트나 음성과 같은 자유로운 형식의 데이터를 구조화하여 분석
- 기계학습과 딥러닝 활용 : 머신러닝, 딥러닝 모델을 기반으로 언어 데이터 학습 및 처리
- 언어별 특화된 접근 필요 : 언어마다 문법과 특성이 다르기 때문에 맞춤형 처리가 필요
- 문맥 및 의미 이해 : 단순 단어 처리뿐만 아니라 문맥과 의미를 파악하는 기술 요구
- 주요 작업 구성 : 텍스트 전처리, 토큰화, 품사 태깅, 문맥 이해 등 다양한 프로세스 포함



1. 거대언어모델(LLM) 정의

1.2 NLP(자연어 처리) ▶ 1.2.2 NLP 과정(1/2)

○ 토큰나이징(Tokenizing)

- ☑ 정의 : 문장을 단어, 형태소, 또는 서브 워드 단위로 나누는 과정
- ☑ 예: “자연어처리” → [“자연“, “어“, “처리“] // 이렇게 쪼개진 단위를 토큰이라 부름

○ 인코딩(Encoding)

- ☑ 토큰나이징된 결과(단어)를 숫자로 변환하는 과정
- ☑ 예: [“자연“, “어“, “처리“] → [101, 2048, 3567]

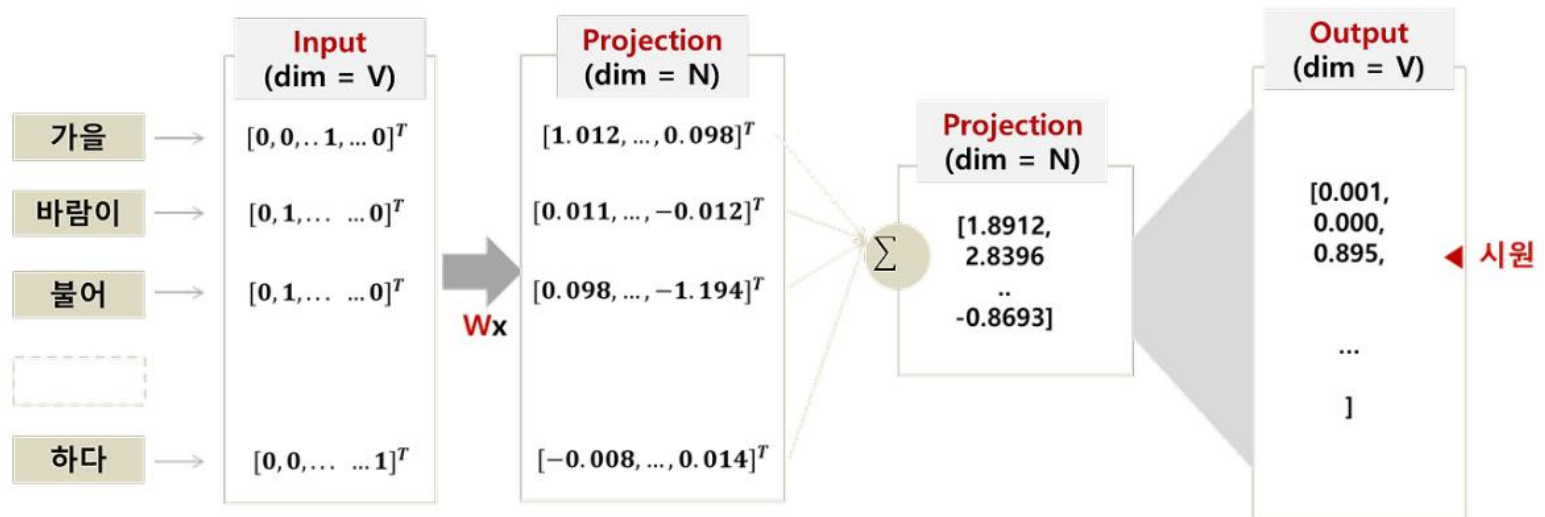
○ 임베딩(Embedding)

- ☑ 숫자로 변환된 단어를 고차원의 실수 벡터로 변환하는 과정
- ☑ 예: [101, 2048, 3567] → [[0.25, -0.1, ...], [0.01, 0.98, ...], [0.5, -0.3, ...]]
- ☑ 벡터의 의미 : 단어 간의 관계(유사성)와 문맥 정보를 표현

1. 거대언어모델(LLM) 정의

1.2 NLP(자연어 처리) ▶ 1.2.2 NLP 과정(2/2)

- ☑ 토큰나이징 : 문장을 단어(또는 글자)로 쪼개는 과정 → 문장을 퍼즐 조각으로 나누기
- ☑ 인코딩 : 퍼즐 조각마다 고유한 번호를 붙임 → 퍼즐 조각에 번호를 매기기
- ☑ 임베딩 : 퍼즐 조각이 다른 퍼즐 조각과의 관계를 이해할 수 있게 벡터를 만들 → 퍼즐 조각이 서로 얼마나 비슷한지 수치화
- ☑ BERT : 퍼즐을 양방향(앞뒤)을 모두 보며 맞추는 사람
- ☑ GPT : 퍼즐을 한 방향(앞에서부터)만 보며 맞추는 사람



2. 트랜스포머(Transformer)

2.1 트랜스포머의 핵심 개념

- ☑ 트랜스포머는 **기존의 순환 신경망(RNN), LSTM 및 합성곱 신경망(CNN)**이 가진 **한계를 극복**하여 LLM 기술핵심 기반이 됨
- ☑ 트랜스포머 기술기반에서 BERT(인코더 사용), GPT(디코더 사용) 방식의 대규모 언어 모델(LLM)으로 발전
- ☑ 100억개 내외의 매개변수의 모델에서 1,700억개의 매개변수(GPT-3)이 폭발적으로 늘면서 초거대 AI모델로 확장

Attention 매커니즘

- 문장의 모든 단어 간의 연관성을 계산하여 중요한 단어에 더 많은 집중(attention)을 함
- 예를 들어, 문장 “고양이가 나무 위로 올라갔다” 에서 ‘고양이’와 ‘올라갔다’ 가 밀접한 관계임을 이해

Self-Attention

- Self-Attention은 문장내의 모든 단어가 서로 얼마나 연관성이 있는 지 계산하는 과정임
- 이를 통해 단어의 순서나 위치에 의존하지 않고도 문장의 전체 의미를 이해
- 예를 들어 “나는 사과를 좋아한다.”라는 문장에서 “사과”라는 단어가 “좋아한다” 라는 단어와 더 밀접하게 관계가 있음을 알아냄

병렬처리

- 기존의 RNN은 데이터를 순차적으로 처리했으나, 트랜스포머는 **병렬처리**가 가능하여 훨씬 더 빠른 학습과 추론 속도를 제공 (ex. RNN : 레고블록을 하나씩 쌓는 과정, 트랜스포머 : 레고블록을 한꺼번에 쌓는 과정)
- 이는 대규모 데이터를 효율적으로 처리할 수 있는 기반이 됨

2. 트랜스포머(Transformer)

2.2 트랜스포머의 구조

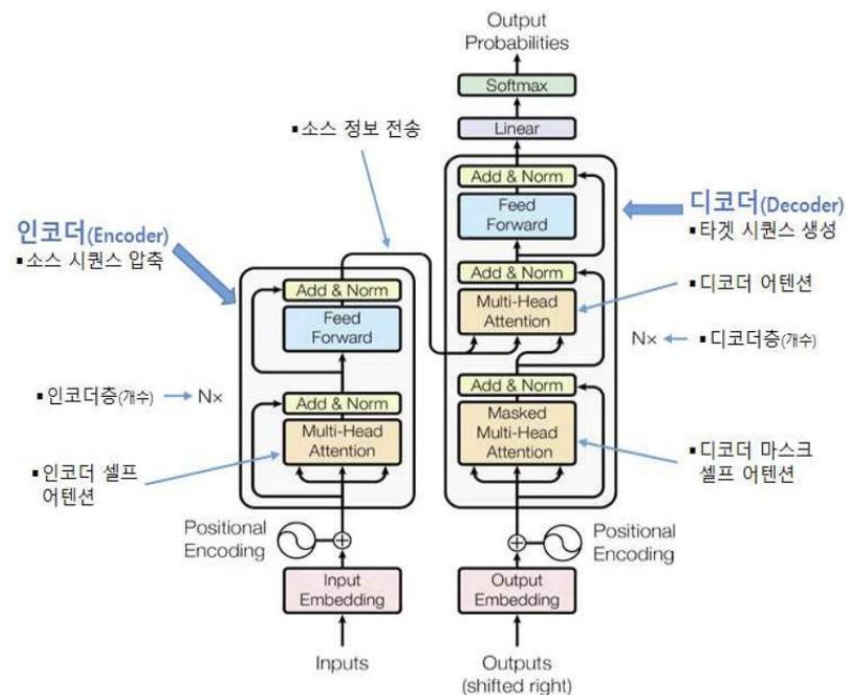
- ☑ 트랜스포머는 인코더(Encoder)와 디코더(Decoder)로 구성됨

인코더 (Encoder)

- 입력 문장을 이해하고, 이를 의미있는 벡터 표현으로 변환하는 역할
- 마치 문장을 다른 언어로 번역하기 전에 원문을 꼼꼼하게 분석하는 과정과 비슷

디코더 (Decoder)

- 인코더에서 생성된 벡터 표현을 바탕으로 새로운 문장을 생성
- 마치 번역된 문장을 만들어내는 과정과 유사



트랜스포머(Transformer) 구조

2. 트랜스포머(Transformer)

2.2 BERT와 GPT 차이점

- ☑ BERT : 문장 전체를 한꺼번에 보고, 각 단어의 역할을 파악하는 탐정에 비유
- ☑ GPT : 주어진 단어를 토대로 이야기를 만들어 나가는 작가에 비유

특징	BERT	GPT
방향성	양방향	단방향
학습 목표	다양한 자연어 처리 태스크	다음 단어 예측
강점	문맥 이해 다양한 자연어처리에 사용	텍스트 생성 능력 우수 챗봇, 번역 등에 사용
단점	모델이 크고 복잡, 학습시간 오래걸림	단방향이어서 문맥이해 능력이 떨어짐

2. 트랜스포머(Transformer)

2.3 트랜스포머 파이프라인

- ☑ 텍스트가 모델이 이해할 수 있는 형태로 전처리 과정을 거침
- ☑ 전처리된 입력이 모델 입력으로 들어감
- ☑ 모델의 예측값이 후처리를 거쳐, 사람이 이해할 수 있는 형태로 반환됨

파이프라인 종류

- feature-extraction: 특징 추출 (텍스트에 대한 벡터 표현 추출)
- fill-mask : 마스크 채우기
- ner : 개체명 인식 (named entity recognition)
- question-answering : 질의응답
- sentiment-analysis : 감정 분석
- summarization : 요약
- text-generation : 텍스트 생성

3. 국내외 거대언어모델(LLM)

○ 해외 LLM

기업	모델	출시일	파라미터 수	특징 및 용도
OpenAI	GPT-4o	2024년 5월	수십억 ~ 수천억	ChatGPT Plus 및 Bing에 활용
Meta	LLaMA 3	2024년 4월	70억 ~ 220억	연구 및 효율성 중점, 오픈소스
Anthropic	Claude 3.5	2024년 6월	1조 3천억	대화 지향, 안전성 강조
Google	Gemini 2.0	2024년 12월	2800억	멀티모달 AI, 이미지 생성 지원
xAI	Grok3	2025년 2월	수백억	고급 대화, 과학 연구, 문제해결

○ 국내 LLM

기업	모델	출시일	파라미터 수	특징 및 용도
네이버	Hyper Clova X	2023년	240억	한글 최적화, 비즈니스 활용
카카오	KoGPT	2024년	130억	한국어 중심 대화 및 생성
LG AI Research	ExaONE	2021년	3000억	AI 연구 및 B2B 솔루션

4. 허깅페이스(Hugging Face)

4.1 허깅페이스 (Hugging Face)란?

- ☑ 자연어 처리(NLP), 컴퓨터 비전, 오디오 처리 등을 위한 머신러닝 모델과 도구를 제공하는 플랫폼
- ☑ 연구자, 개발자, 데이터 과학자들이 AI모델을 쉽게 접근하고 사용할 수 있도록 지원하며, 커뮤니티 중심으로 발전
- ☑ 주요 서비스 : 트랜스 포머 라이브러리, 데이터셋, 인터페이스 API, 스페이스(스테이블디퓨전, DALL-E 등 여러모델 체험가능)

허깅페이스 허브

- 사전 학습된 AI 모델 저장소로, NLP, 컴퓨터 비전, 오디오 모델이 포함되어 있음
- 누구나 모델을 업로드하거나 다운로드 가능
- 모델 외에도 데이터셋과 파이프라인 제공

라이브러리 제공

- transformers : NLP 및 다양한 도메인에서 사용할 수 있는 사전 학습된 모델
- datasets : 다양한 데이터셋을 로드하고 전처리할 수 있는 도구
- tokenizers : 효율적인 토큰나이징(Tokenizing) 처리
- diffusers : 이미지 생성 및 안정적 확산 모델 구현
- * cf. 토큰나이징 : 컴퓨터가 이해할 수 없는 언어를 컴퓨터가 이해할 수 있는 숫자로 바꾸는 과정

인터페이스

- 웹 기반의 허깅페이스 허브(Hugging Face Hub)와 API를 통해 사용 가능
- 직접 파이썬 코드로 실행 가능

4. 허깅페이스(Hugging Face)

4.2 허깅페이스 (Hugging Face) 사용방법

- ☑ 필수 설치 : Python (3.7버전 이상), Jupyter Notebook 또는 Colab 환경 설정
- ☑ Hugging Face Hub 웹사이트 : <https://huggingface.co>
- ☑ 파이썬 환경에서 설치 및 활용 : `pip install transformers datasets`
- ☑ 사전 학습 모델 로드 및 사용 예시

```
from transformers import pipeline

# 감정 분석 파이프라인 로드
sentiment_pipeline = pipeline("sentiment-analysis")

# 텍스트 분석
result = sentiment_pipeline("Hugging Face is amazing!")
print(result)

결과값 : [{'label': 'POSITIVE', 'score': 0.999886155128479}]
```

5. 랭체인(LangChain)

5.1 랭체인 (Langchain) 이란?

- ☑ LLM을 활용한 **응용 프로그램을 개발할 수 있도록 돕는 프레임워크**
- ☑ LangChain은 LLM과 다양한 도구(데이터베이스, API, 파일 시스템 등)를 통합하여 어플리케이션을 만들 수 있도록 설계됨
- ☑ 주로 챗봇, 문서 요약, 질문 응답 시스템 등을 구현할 때 사용

컴포넌트 기반

- LangChain은 여러 모듈(컴포넌트)로 나뉘어 있으며, 필요에 따라 선택적으로 사용가능

[주요 컴포넌트]

- LLM Wrappers : OpenAI, Hugging Face, Cohere 등 다양한 LLM과 통합 가능
- Prompt Templates : LLM의 프롬프트를 체계적으로 구성하고 관리
- Memory : 대화 기록 및 상태를 관리하여 컨텍스트를 유지
- Chains : 여러 작업(프롬프트, API호출 등)을 연결하여 워크플로를 구성
- Agents : 외부 도구(API, 데이터베이스 등)와의 상호작용을 통해 문제를 해결

기타 특징

- 다양한 LLM 지원 : OpenAI GPT-4, Hugging Face 모델, Anthropic Claude 등
- 외부 데이터와 통합 : 데이터베이스(SQL, MongoDB 등), 파일(PDF, csv, xls) 등
- 확장성과 유연성 : 단순한 사용부터 복잡한 파이프라인까지 다양한 요구사항에 맞게 설계 가능

5. 랭체인(LangChain)

5.2 랭체인 (LangChain) 사용방법

- ☑ 필수 설치 : Python (3.8 버전 이상), Jupyter Notebook 또는 Colab 환경 설정
- ☑ 필수 패키지 설치 : `pip install langchain openai`, `pip install langchain_community`
- ☑ llm 모델 설치 : ollama 설치 후 해당하는 모델 사이트에서 설치 (gemma2, llama3, mistral 등)
- ☑ 사전 학습 모델 로드 및 사용 예시

```
# ollama의 Langchain 커뮤니티 라이브러리에서 ollama llm 클래스 import
from langchain_community.llms import Ollama
```

```
# llm 모델 지정
llm = Ollama(model = "gemma2")
```

```
# invoke() 메서드를 사용해 모델에 프롬프트 전달 및 응답 요청
llm.invoke("안녕, 간단하게 소개해줘.")
```

결과값 : 저는 Google DeepMind에서 훈련된 대규모 언어 모델입니다.

5. 랭체인(LangChain)

5.3 Ollama (1/3)

○ Ollama 정의 및 특징

- ☑ Ollama는 AI모델과 관련된 도구 및 서비스를 제공하는 플랫폼으로, LLM의 배포 및 활용에 중점을 둠
- ☑ LLM 모델인 ChatGPT가 보통 클라우드 API접근 및 유료이지만, Ollama는 오픈소스 무료 LLM을 실행할 수 있게 지원

Ollama 특징

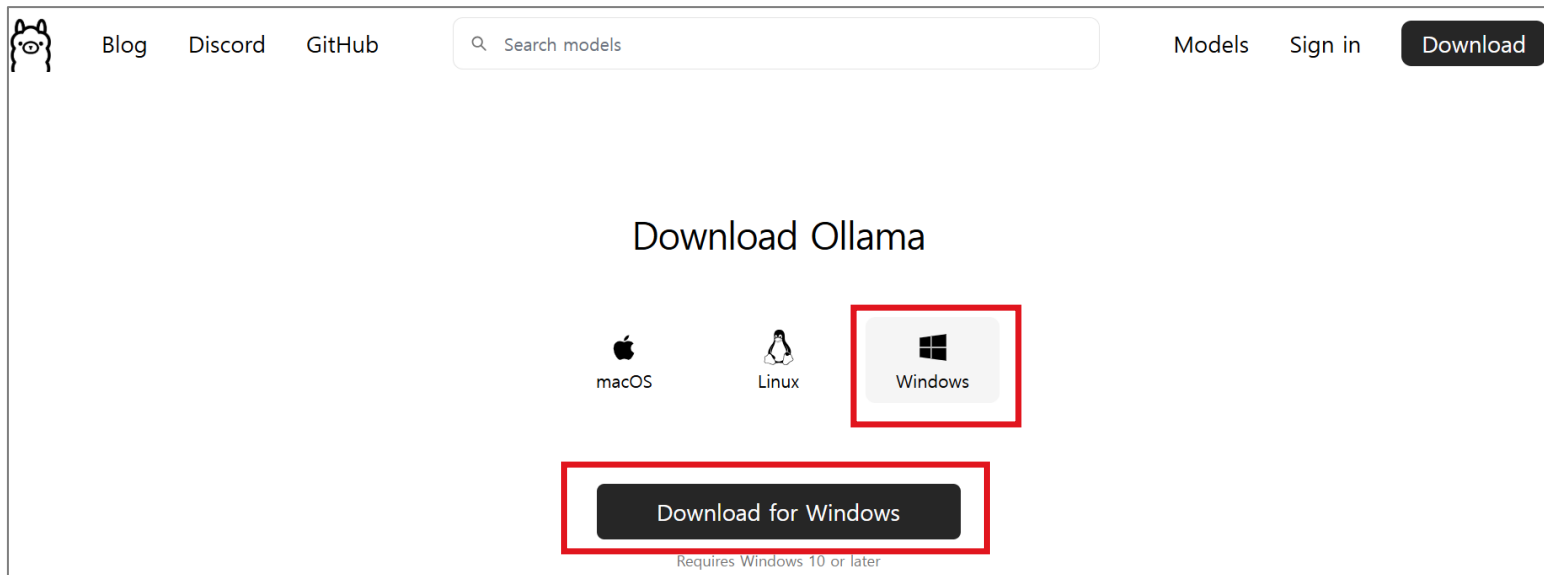
- 로컬 실행 : Ollama는 클라우드 서비스에 의존하지 않고도 LLM을 로컬에서 실행할 수 있게 지원
- 데이터 프라이버시 : 로컬에서 모델을 실행하기 때문에 사용자 데이터가 외부 서버로 전송이 안됨
- 성능 최적화 : 로컬 하드웨어의 성능을 최대한 활용 (특히 GPU 활용 시 높은 성능을 얻을 수 있음)
- 비용 절감 : 클라우드 사용에 따른 서비스 비용을 절감할 수 있음
- 모델 커스터마이징 : 사용자는 자신의 요구에 맞게 모델을 커스터마이징 가능
- 오프라인 사용 : 인터넷 연결 없이도 모델을 사용 가능

5. 랭체인(LangChain)

5.3 Ollama (2/3)

○ Ollama 설치

- ☑ 먼저, Ollama 설치 : Ollama 공식 사이트 (<https://ollama.com/download/windows>)
- ☑ Ollama 사이트에서 Download for Windows 클릭



5. 랭체인(LangChain)

5.3 Ollama (3/3)

○ sllm(smal large language model) 설치

☑ 윈도우 cmd(터미널 창)에서 gemma2 다운로드 명령어 : ollama pull gemma2

```
pulling manifest
pulling 6a0746a1ec1a... 100%
pulling 4fa551d4f938... 100%
pulling 8ab4849b038c... 100%
pulling 577073ffcc6c... 100%
pulling 3f8eb4da87fa... 100%
verifying sha256 digest
writing manifest
removing any unused layers
success
```

4.7 GB
12 KB
254 B
110 B
485 B

☑ 설치 이후 확인 명령어 : ollama list

```
C:\Users\ahnda>ollama list
```

NAME	ID	SIZE	MODIFIED
llava:13b	0d0eb4d7f485	8.0 GB	4 weeks ago
gemma2:latest	ff02c3702f32	5.4 GB	4 weeks ago

6. RAG(검색증강생성)

6.1 RAG (검색증강생성) 이란?

- ☑ 단순히 모델이 학습된 데이터에 의존하는 것이 아니라, 최신 정보나 특정 데이터셋에 기반한 응답을 생성
- ☑ 외부지식(문서, 데이터베이스 등)을 검색한 뒤, 이를 활용해 더 정확하고 풍부한 응답을 생성하는 방법

RAG의 작동 원리

[검색단계]

- 질문(쿼리)을 기반으로 외부 데이터 소스에서 관련된 문서를 검색
- 예시 : 데이터베이스, PDF 파일, 웹 크롤링 데이터, 벡터 데이터베이스 등

[생성단계]

- 검색된 정보를 LLM에 입력(prompt)으로 제공하여 응답을 생성
- LLM은 검색된 정보를 기반으로 컨텍스트를 이해하고 응답을 작성

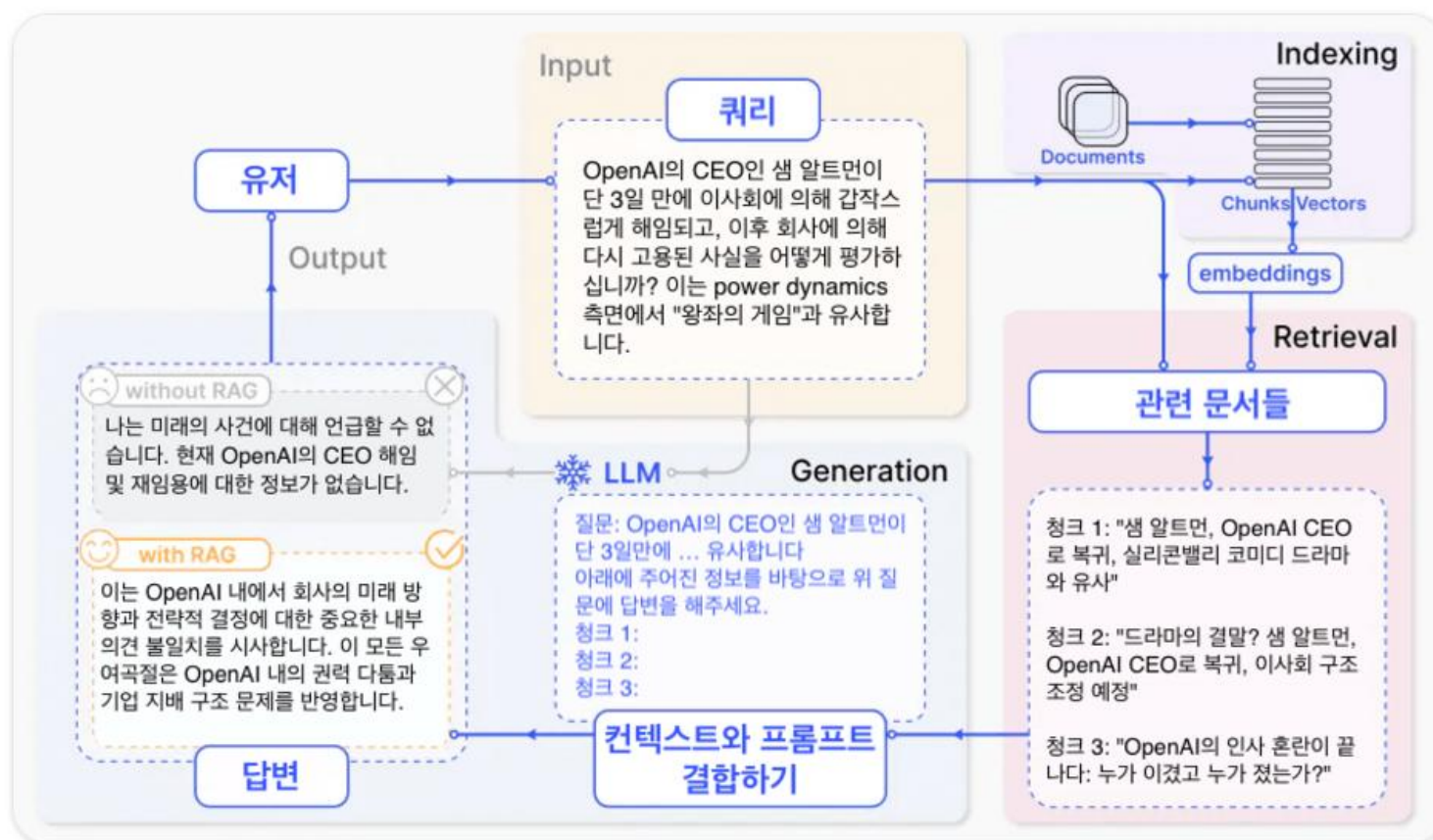
RAG 주요특징

- 외부 데이터 활용 : 모델 학습 데이터 외에 정보 소스에서 최신 정보를 가져와서 사용 가능
- 효율적인 검색과 생성 결합 : 검색시스템과 언어 모델을 결합해 단일 LLM보다 더 높은 정확도 제공
- 최신성과 확장성 : 최신 데이터나 업데이트된 정보를 즉시 반영할 수 있음
- 모듈형 구조 : 검색과 생성이 분리되어 있어 각각 독립적으로 최적화 가능
 - 검색 시스템 : 벡터 데이터베이스(FAISS, Chroma 등)
 - 생성모델 : GPT, BERT 외 sllm(gemma2, llama3 등) 다양한 LLM 사용 가능

6. RAG(검색증강생성)

6.2 RAG 구성 요소

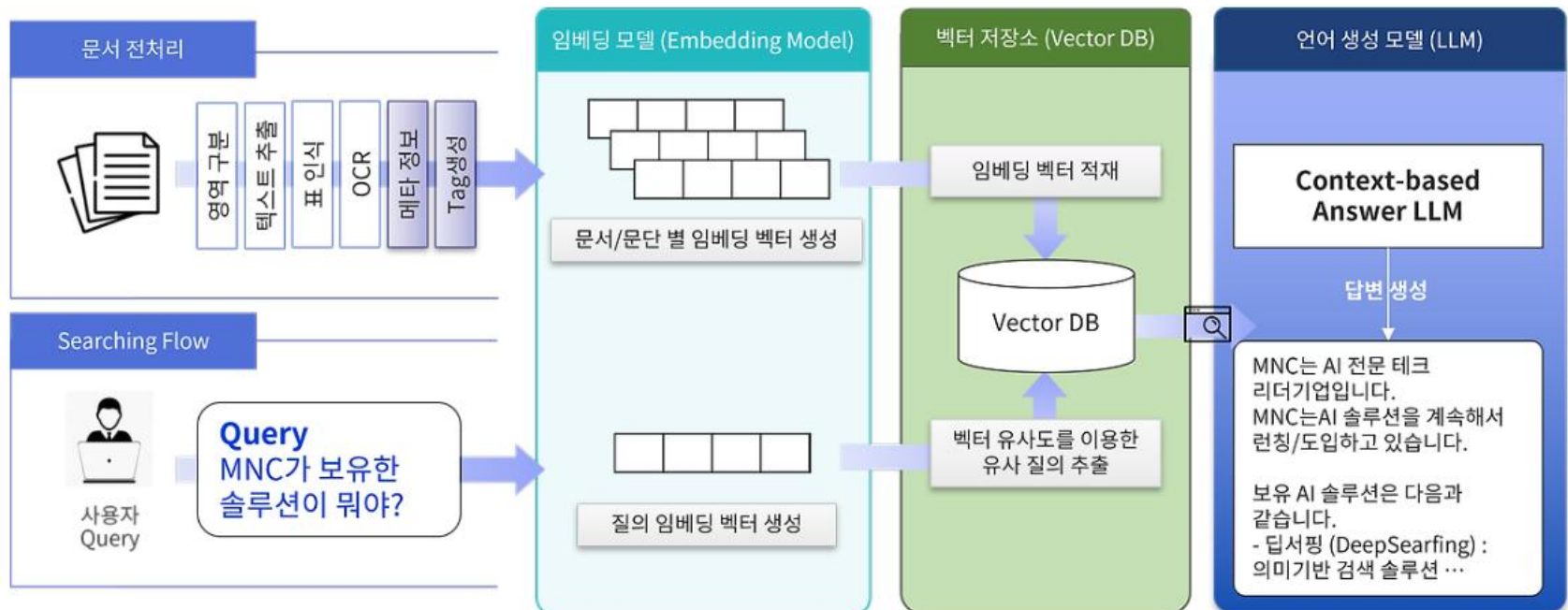
- ☑ 자연어 처리(NLP), 컴퓨터 비전, 오디오 처리 등을 위한 머신러닝 모델과 도구를 제공하는 플랫폼
- ☑ 연구자, 개발자, 데이터 과학자들이 AI모델을 쉽게 접근하고 사용할 수 있도록 지원하며, 커뮤니티 중심으로 발전



6. RAG(검색증강생성)

6.3 RAG 파이프라인

- ☑ 사용자의 질문을 벡터로 변환하고, 이를 기반으로 데이터베이스 내에서 가장 연관된 문서나 정보를 찾아냄
- ☑ LLM은 그 후 이 문서들을 기반으로 사용자의 질문에 가장 적절한 답변 생성



6. RAG(검색증강생성)

6.4 활용 사례

문서 기반 질의응답

- PDF, 워드 문서, 데이터베이스 등에서 정보를 검색해 정확한 답변 제공
- 예시 : 회사 내부 보고서를 바탕으로 질문 응답

실시간 정보제공

- 최신 뉴스나 제품 정보를 검색해 사용자의 질문에 응답
- 예시 : “오늘 날씨 어때?” 또는 “최신 아이폰 스펙 알려줘.”

헬프데스크/ 챗봇

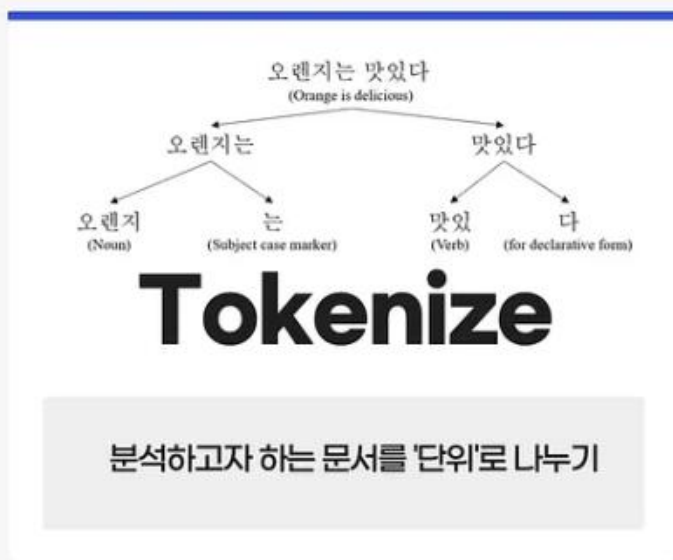
- 고객 지원을 위해 FAQ나 매뉴얼 데이터 검색 후 응답
- 예시 : 제품 사용 방법 안내

6. RAG(검색증강생성)

6.5 RAG의 핵심 기술 ▶ 6.4.1 자연어 처리 과정

- ☑ AI모델은 숫자 형태의 입력과 출력만을 처리할 수 있어서, 텍스트를 AI모델이 이해할 수 있는 숫자 형태로 변환해야 함
- ☑ 이를 위해 자연어(NLP) 과정을 통해 시가 자연어로 이해할 수 있도록 하는데, **Tokenize, Embedding 과정 필요**

자연어 처리의 과정



	long	back	none	human	gender	major	verb	place
cat →	0.6	0.9	0.1	0.4	-0.7	-0.3	-0.2	
kitten →	0.5	0.8	-0.1	0.2	-0.6	-0.5	-0.1	
dog →	0.7	-0.1	0.4	0.3	-0.4	-0.1	-0.3	
houses →	-0.8	-0.4	-0.5	0.1	-0.9	0.3	0.8	

Embedding

분석의 단위를, 컴퓨터가 이해할 수 있는 '숫자'로 만들기

6. RAG(검색증강생성)

6.5 RAG의 핵심 기술 ▶ 6.4.2 토큰나이저(Tokenizer)

- ☑ 원본 문장이나 문단이 AI 모델에 입력되기 전에 더 작은 조각으로 쪼개지는 과정
- ☑ 이때 조각을 토큰(Token), 모델을(Tokenizer)라고 함
- ☑ 사용되는 토큰나이저에 따라 하나의 토큰이 단어가 될 수도 있고, 단어의 일부 조각이 될 수도 있음
- ☑ 토큰 하나로부터 추출한 임베딩을 단어 임베딩이라고 함

Tokenizer

The GPT family of models process text using **tokens**, which are common sequences of characters found in text. The models understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens.

You can use the tool below to understand how a piece of text would be tokenized by the API, and the total count of tokens in that piece of text.

GPT-3 Codex

Syncly uses AI to automatically categorize and analyze your communication with customers, helping customer-facing teams understand most urgent customer issues at scale.

Clear Show example

Tokens	Characters
28	168

Syncly uses AI to automatically categorize and analyze your communication with customers, helping customer-facing teams understand most urgent customer issues at scale.

TEXT TOKEN IDS

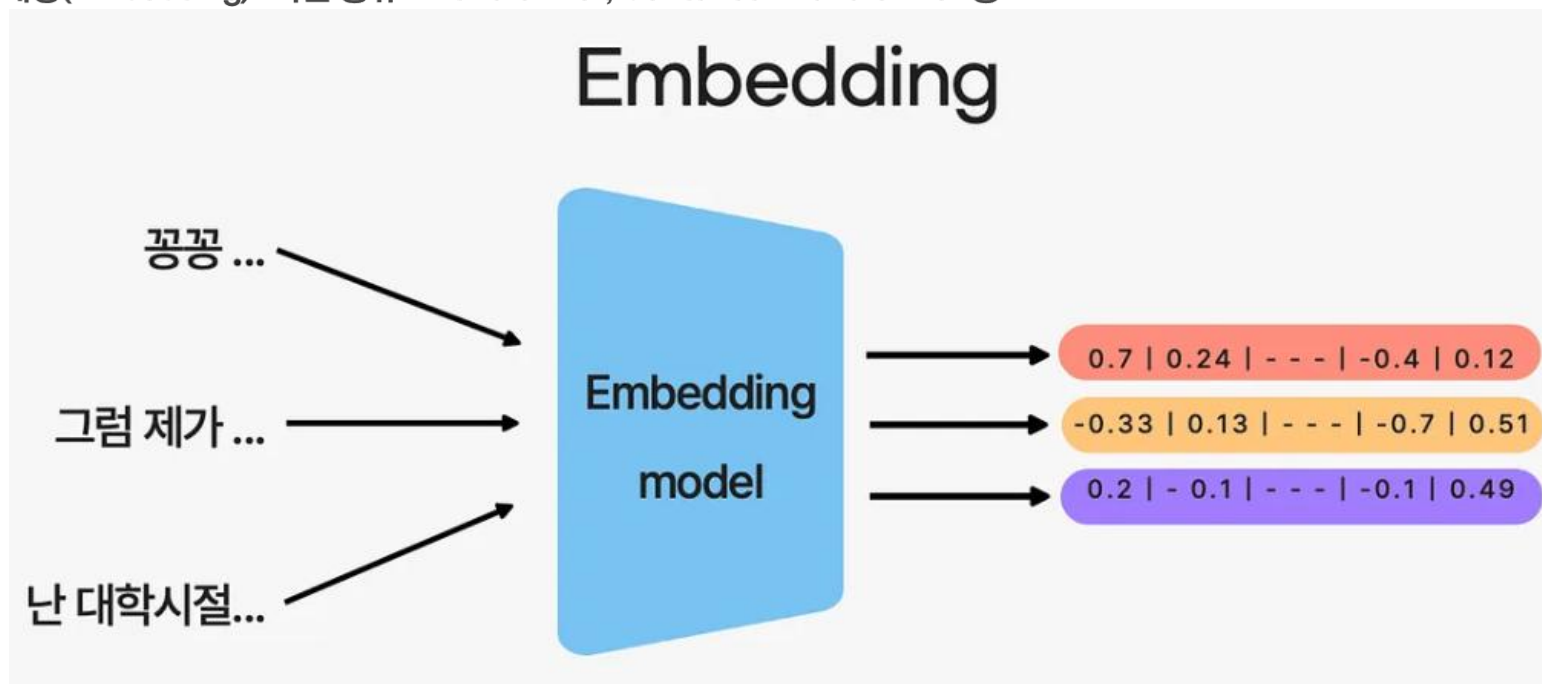
A helpful rule of thumb is that one token generally corresponds to ~4 characters of text for common English text. This translates to roughly ¾ of a word (so 100 tokens ~ 75 words).

If you need a programmatic interface for tokenizing text, check out our [tiktoken](#) package for Python. For JavaScript, the [gpt-3-encoder](#) package for node.js works for most GPT-3 models.

6. RAG(검색증강생성)

6.5 RAG의 핵심 기술 ▶ 6.4.3 임베딩(Embedding)

- ☑ AI모델이 이해할 수 있도록 텍스트를 실수 벡터 형태로 표현한 결과를 의미
- ☑ 특정한 단어, 문장 또는 문서를 임베딩 생성 모델에 입력하면, 일정 수의 실수로 구성된 벡터가 출력
- ☑ 벡터(Vector) 유사성 검색 라이브러리 : FAISS, Chroma 등
- ☑ 임베딩(Embedding) 지원 종류 : Transformer, Sentence-Transformer 등



6. RAG(검색증강생성)

6.6 RAG 예제 (1/2)

- ☑ 문서로드 : TextLoader를 사용하여 텍스트 데이터를 로드함
- ☑ 테스트를 위한 텍스트 예시 데이터를 dataset 폴더에 저장
- ☑ TXT파일 외에도 PDF, CSV와 같은 다른 형식의 데이터를 사용하는 경우, 해당 형식에 맞는 Loader 사용
- ☑ 벡터화 : HuggingFaceEmbeddings를 사용하여 문서의 텍스트를 임베딩(벡터화)
- ☑ 벡터 데이터는 FAISS(Vector Database)에 저장되며, 검색 단계에서 활용됨

```
from langchain.chains import RetrievalQA
from langchain.document_loaders import TextLoader
from langchain.vectorstores import FAISS
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.llms import Ollama

# 1. 데이터 로드
loader = TextLoader("./dataset/history.txt") # 텍스트 파일 로드
documents = loader.load() # 문서 로드

# 2. 벡터 임베딩 생성 (Hugging Face 사용)
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vectorstore = FAISS.from_documents(documents, embeddings)
```

6. RAG(검색증강생성)

6.6 RAG 예제 (2/2)

- ☑ 검색기 설정 : retrieve는 검색 단계에서 관련 문서를 반환하는 역할을 함
- ☑ Ollama 기반 LLM 사용 : Ollama의 Gemma2 모델을 로컬에서 실행하기 위해 base_url 설정
- ☑ 질문 실행 : 사용자가 입력한 질문에 대해 검색된 문서를 호라용하여 LLM이 응답

```
# 3. 검색기(retriever) 설정
retriever = vectorstore.as_retriever()

# 4. Ollama Gemma2 모델 초기화
llm = Ollama(model="gemma2", base_url="http://localhost:11434") # Ollama 서버 설정

# 5. RAG 체인 구성
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)

# 6. 질문 실행
query = "고조선은 언제 설립되었는지 알려줘."
response = qa_chain.run(query)
print("Response:", response)

결과값 : 고조선은 기원전 2333년 단군왕검에 의해 세워졌다고 전해집니다.
```